

■ CLOUD COMPUTING · DEVOPS

\$ docker run fundamentals

Docker Fundamentals & Containerization

Lec by Imran Sayyed

2.5 hour session

For first-time learners

Six steps from problem to container

01

Foundations

Why deployment breaks

02

VMs vs Containers

Two ways to isolate

03

Docker Architecture

Client, engine, registry

04

Images & Containers

Pull, run, manage

05

Dockerfile

Build your own image

06

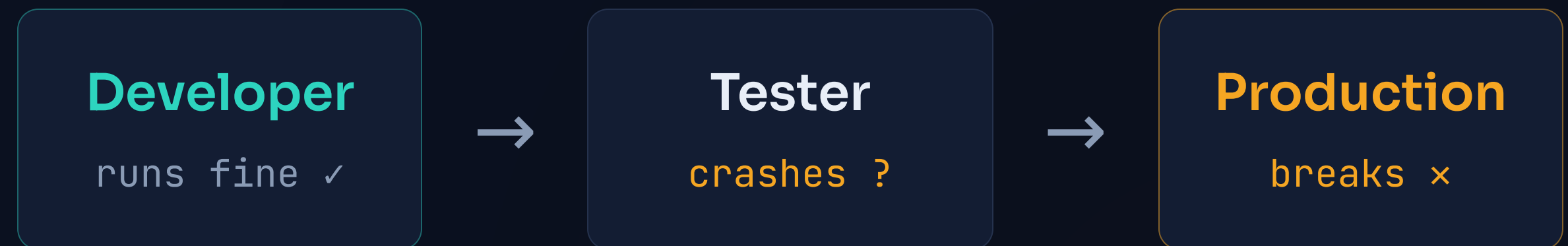
Cloud & Wrap-up

Real-world & review

“But it works on my machine”

The same code behaves differently across environments because the surroundings are never identical.

- ✗ Different OS versions
- ✗ Missing dependencies
- ✗ Mismatched config



Package the app with everything it needs

consistent

Same everywhere

Runs identically on a laptop, a server, or the cloud.

portable

Ship as one unit

Code, runtime, libraries, and config travel together.

efficient

Light & fast

Starts in seconds and shares the host kernel.

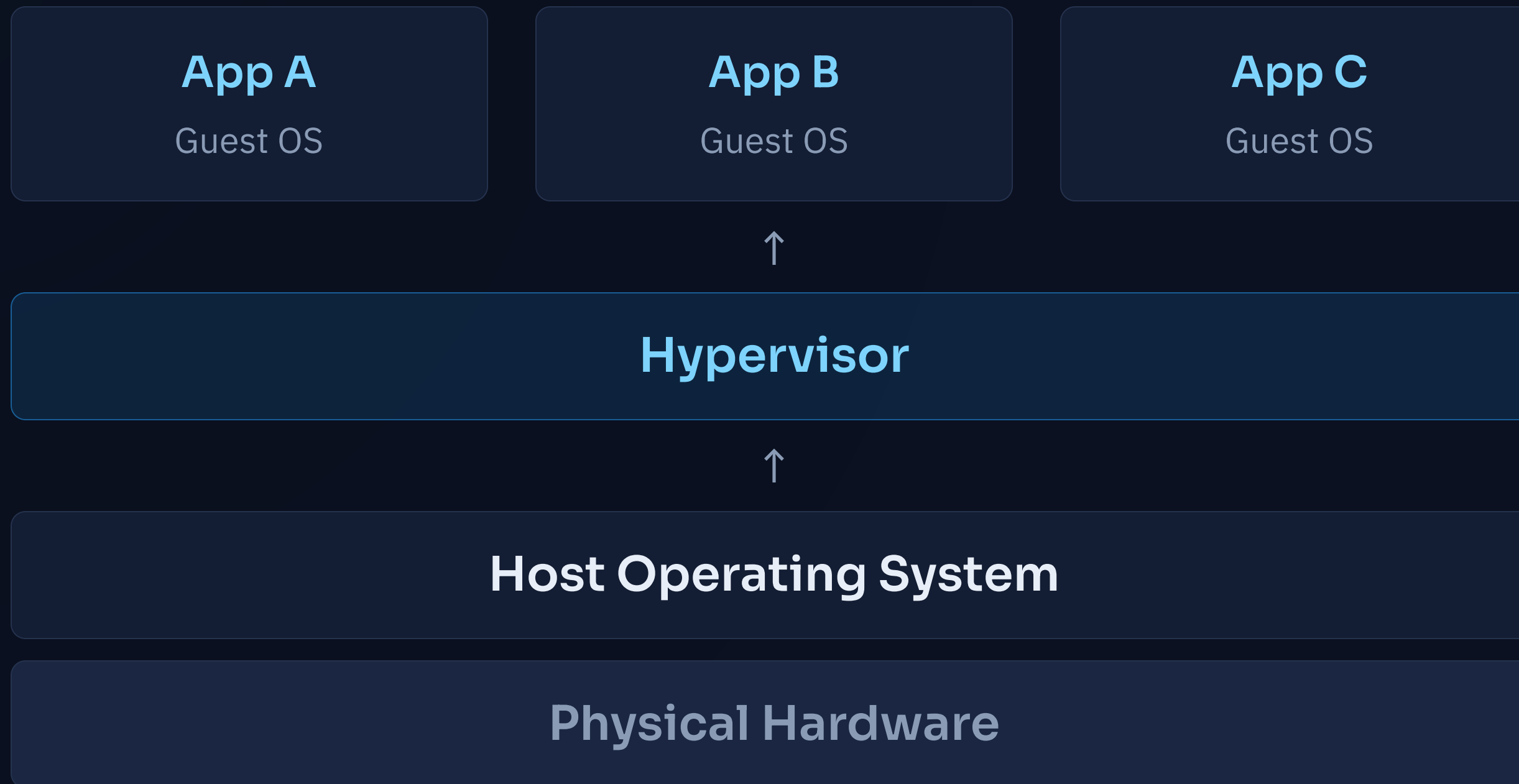
One app, one server

Apps were installed straight onto physical machines, tied to that server's exact setup.

- Slow to provision new servers
- Hard to scale or move
- Wasted, idle capacity



Each VM carries its own full OS



Powerful, but heavy

01 Large in size

Each VM bundles a full OS — often many gigabytes.

02 Slow to boot

Starting a guest OS takes minutes, not seconds.

03 Resource overhead

Every VM reserves its own CPU and memory.

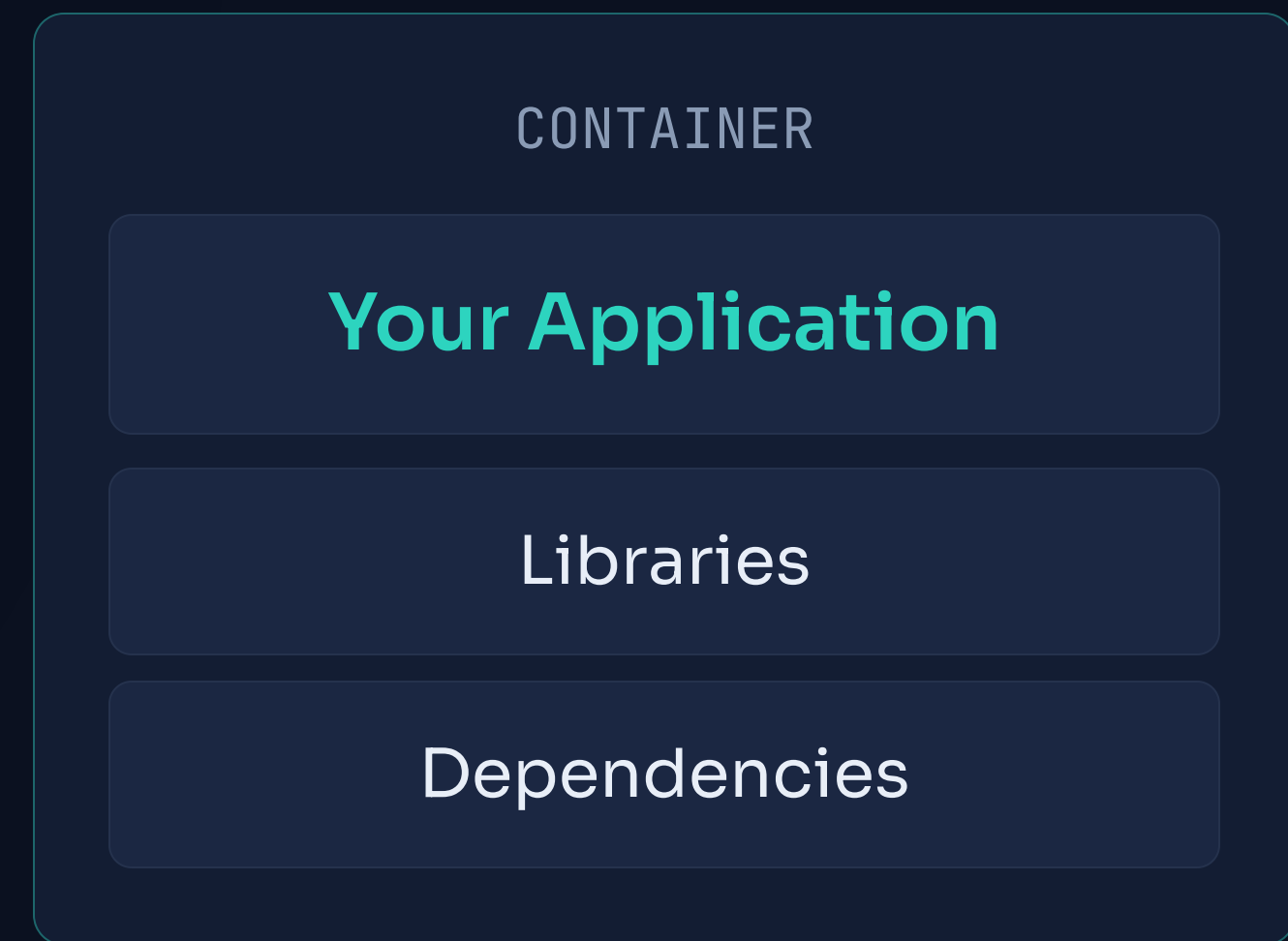
04 Fewer per host

A single server runs only a handful of VMs.

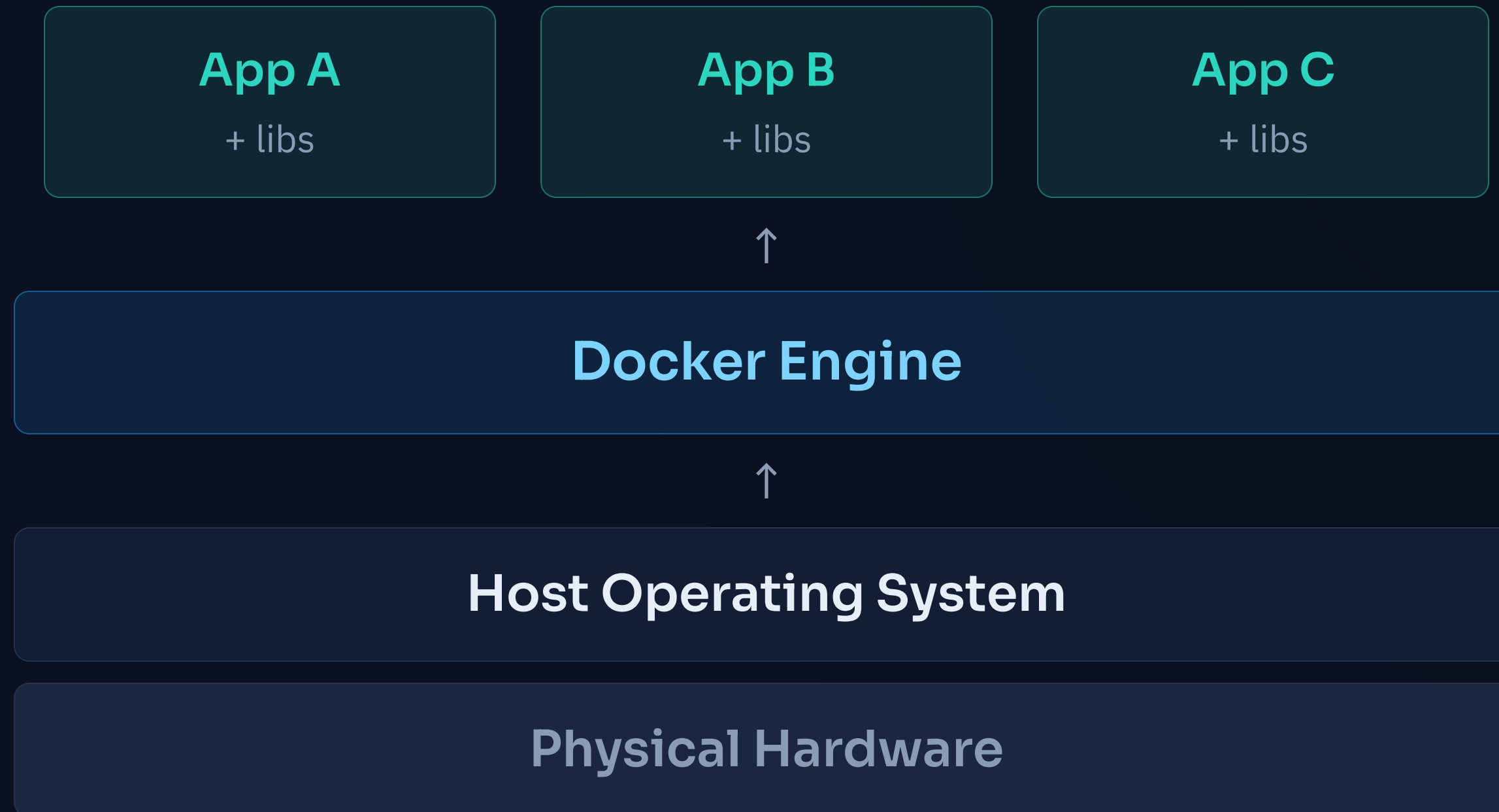
Isolation without a full OS

A container is a lightweight, isolated package that holds an app and its dependencies — sharing the host's kernel instead of shipping a whole operating system.

- ✓ Starts in seconds
- ✓ Megabytes, not gigabytes
- ✓ Many per host



One shared kernel, many containers



Same goal, very different weight

Virtual Machines

Isolation	Full OS per VM
Size	Gigabytes
Startup	Minutes
Overhead	High
Per host	Few

Containers

Isolation	Shared kernel
Size	Megabytes
Startup	Seconds
Overhead	Low
Per host	Many

The standard way to run containers

Docker is an open platform that builds, ships, and runs applications inside containers — turning containerization into a simple, repeatable workflow.

- Build images from a recipe
- Ship them through a registry
- Run anywhere Docker is installed

DOCKER

Build - Ship - Run

any environment

The moving parts you will use

`docker CLI`

Docker Client

The commands you type to control Docker.

`dockerd`

Docker Engine

The daemon that builds and runs containers.

`images`

Images & Containers

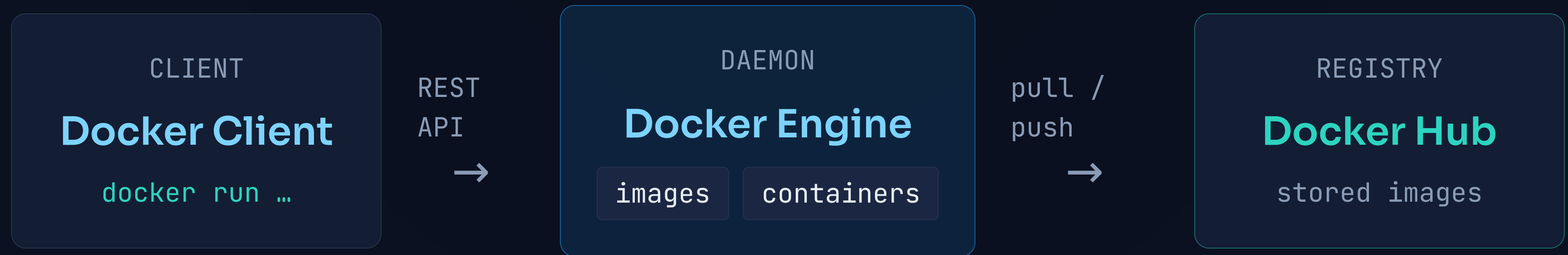
Templates and their running instances.

`registry`

Docker Hub

The cloud registry that stores images.

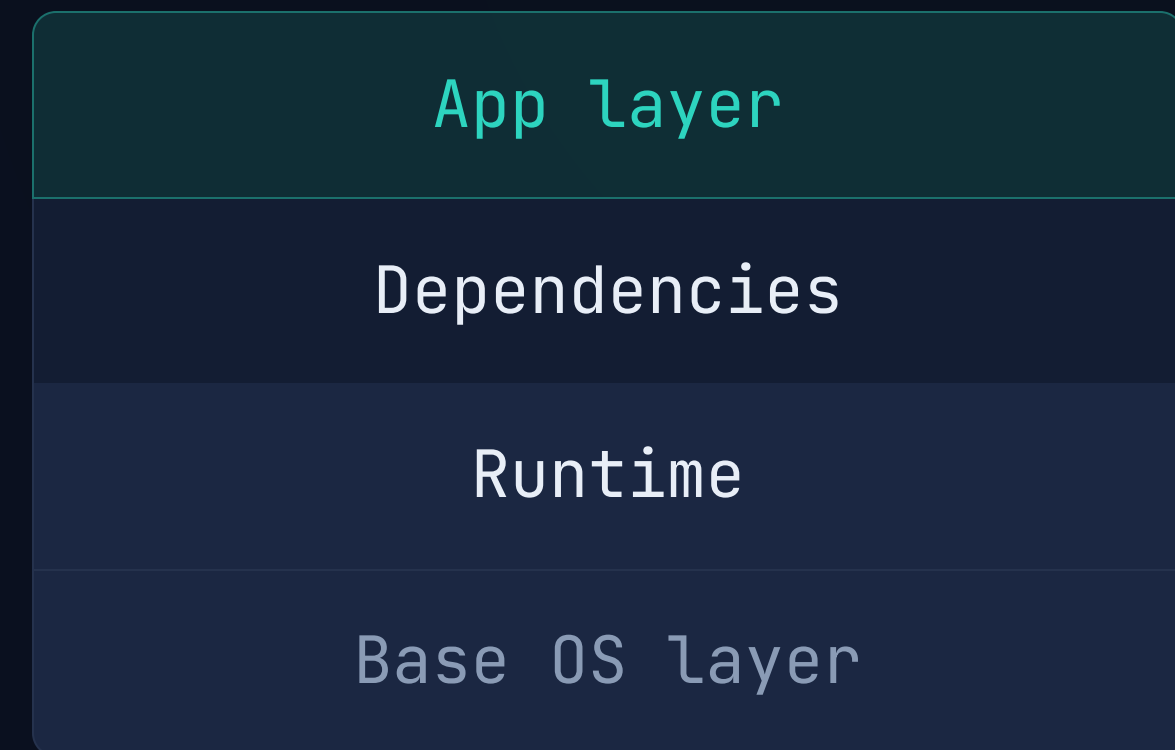
Client, daemon, and registry



A read-only template

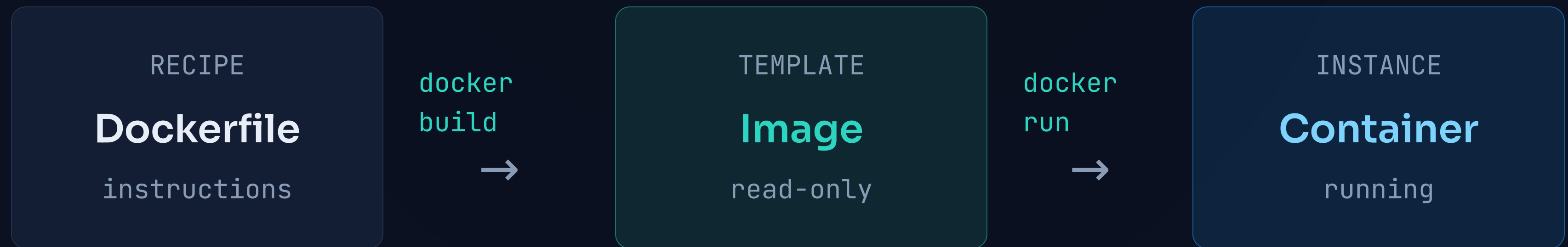
An image is a packaged blueprint of your app — built in stacked layers and frozen so every container starts from the exact same state.

- ✓ Read-only and reusable
- ✓ Built from layers
- ✓ Shared via a registry



stacked, read-only layers

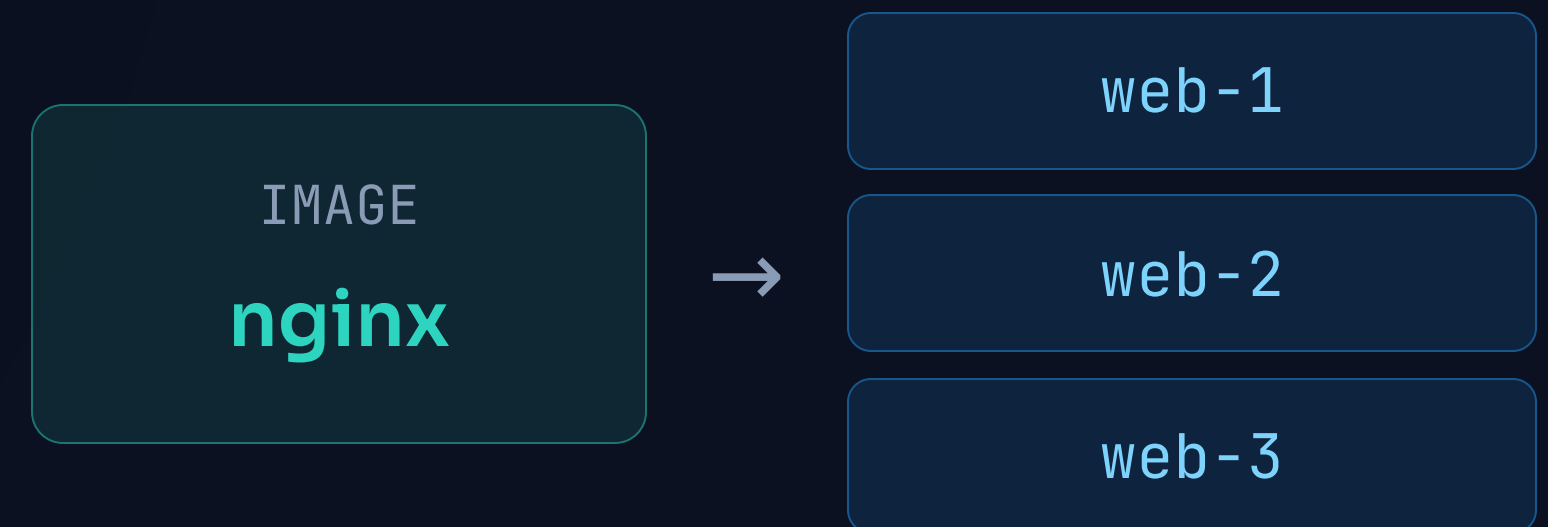
From recipe to running app



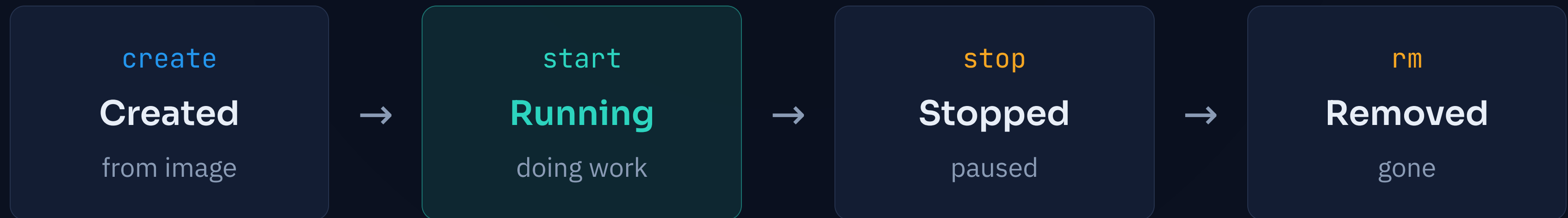
A running instance of an image

If an image is the recipe, a container is the finished dish — a live, isolated process created from the image with its own writable layer on top.

- Created from an image
- Isolated and writable
- Many from one image



Create, run, stop, remove



The public library of images

Docker Hub is a cloud registry where official and community images are published, searched, and pulled.

- Official, trusted images
- Versioned with tags
- Free to pull and use

hub.docker.com	Q search images
nginx	OFFICIAL · 1B+ pulls
node	OFFICIAL · 1B+ pulls
postgres	OFFICIAL · 1B+ pulls

Download an image from the registry

`docker pull` fetches an image to your machine, layer by layer.

- Defaults to the latest tag
- Cached layers are reused

● ● ● bash

```
$ docker pull nginx
```

```
Using default tag: latest
```

```
latest: Pulling from library/nginx
```

```
a2abf6c4d29d: Pull complete
```

```
9b48060f404d: Pull complete
```

```
Status: Downloaded newer image for nginx:latest
```

Turn an image into a live container

`docker run` creates and starts a container from an image in one step.

- Pulls the image if missing
- `-d` runs in the background

bash

```
$ docker run -d -p 8080:80 nginx
3f9a1c7b2e64d8a0c1...

# container started in background

$ docker ps
CONTAINER ID  IMAGE  STATUS
3f9a1c7b2e64  nginx  Up 3 seconds
```

Your first container

`hello-world` is a tiny image that proves your Docker install works end to end.

- Pulls the image
- Runs and prints a message
- Exits immediately

● ● ● bash

```
$ docker run hello-world
```

```
Unable to find image locally
```

```
latest: Pulling from hello-world
```

```
Hello from Docker!
```

```
This message shows your install  
is working correctly.
```

Run a real web server

bash

```
$ docker run -d \
  -p 8080:80 nginx
7c2b9f1a...
# nginx now serving on
# host port 8080
```



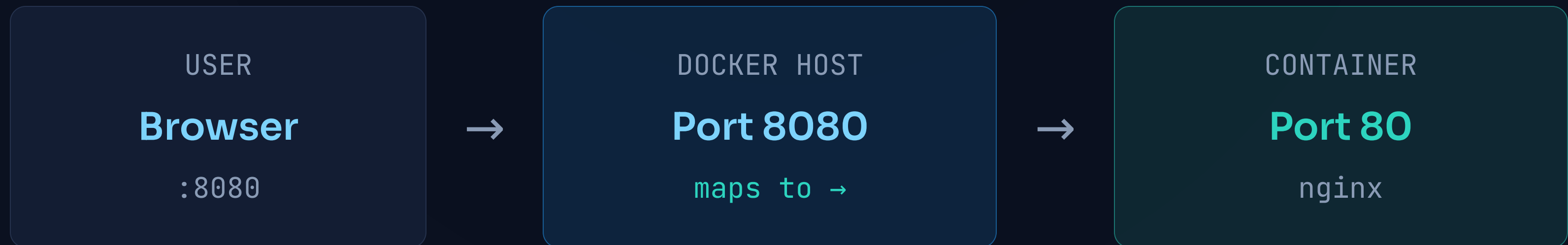
localhost:8080

Welcome to nginx!

If you see this page, the nginx web server is
working.

Connect the outside to the inside

```
-p 8080:80 # host : container
```



How containers talk

`bridge`

Default network

Containers on one host share a private bridge.

`name`

Service discovery

Containers reach each other by container name.

`-p`

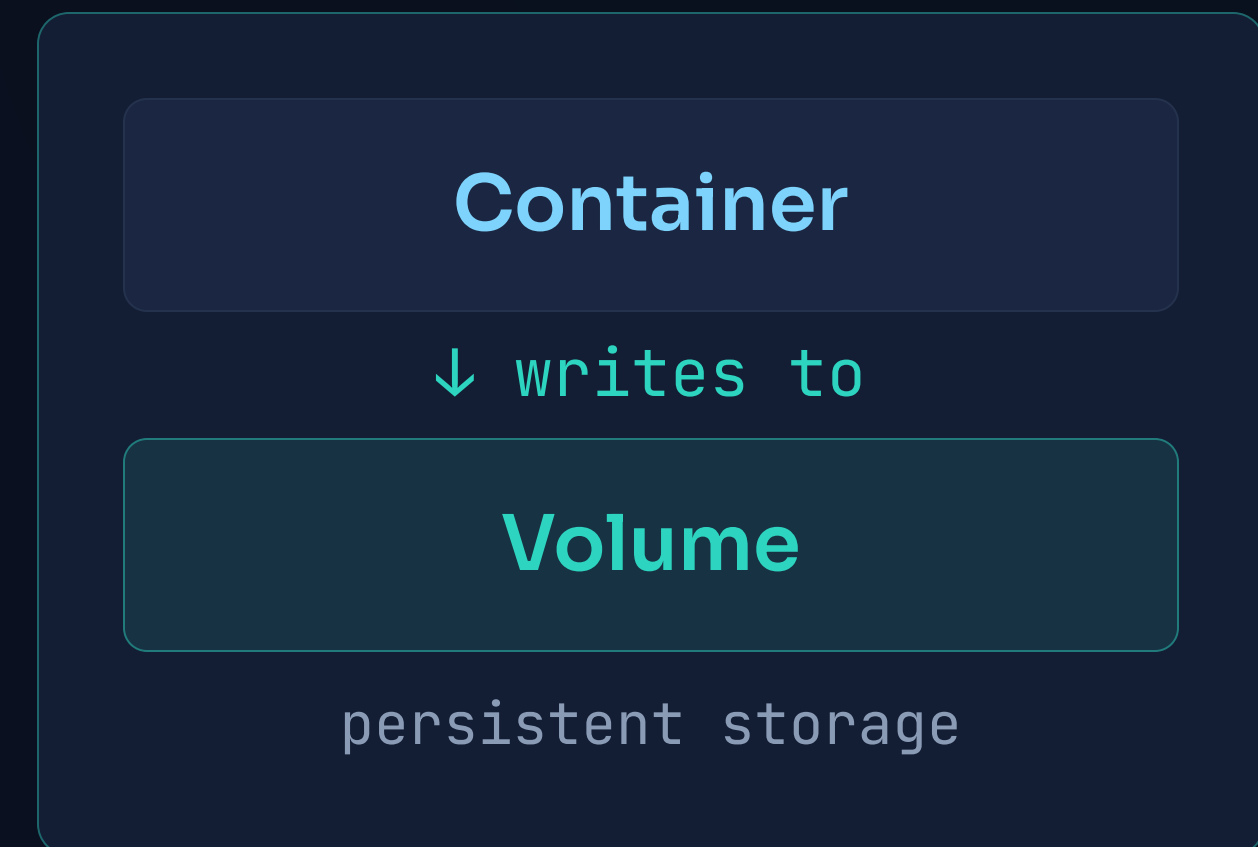
Published ports

Map a port to expose a container to the outside.

Storage that outlives the container

By default, anything written inside a container disappears when it is removed. A volume is managed storage that lives outside the container's writable layer.

- ✓ Survives restarts and removal
- ✓ Shareable between containers
- ✓ Managed by Docker



Containers are disposable, data is not

Without a volume

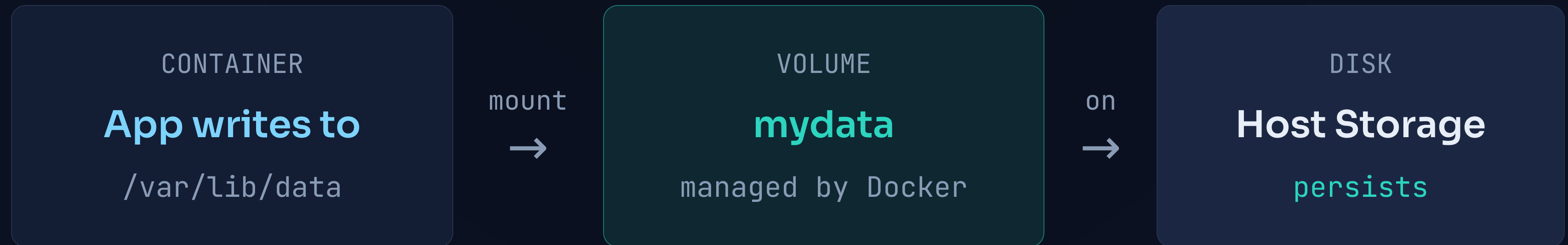
- ✗ Data lives in the container
- ✗ Removed with the container
- ✗ Nothing to recover

With a volume

- ✓ Data stored outside
- ✓ Survives container removal
- ✓ Reattach to a new container

One volume, mounted into a container

```
-v mydata:/var/lib/data
```



The everyday toolkit

`docker pull`

Download an image

`docker run`

Start a container

`docker ps`

List containers

`docker stop`

Stop a container

`docker rm`

Remove a container

`docker images`

List local images

Get an image locally

```
docker pull <image>:<tag>
```

- Downloads from Docker Hub
- Tag defaults to `latest`
- Reuses cached layers

```
$ docker pull python:3.12  
3.12: Pulling from library/python  
f1c2e9a7...: Pull complete  
Downloaded python:3.12
```

Create and start a container

```
docker run [opts] <image>
```

- d run in the background
- p publish a port
- name name the container

```
$ docker run -d \  
  --name web \  
  -p 8080:80 nginx  
  
a1b2c3d4...  
  
# running as "web"
```

See what is running

```
docker ps [-a]
```

- Lists running containers
- a shows stopped ones too
- Shows IDs, status, ports

```
$ docker ps
```

```
CONTAINER ID  IMAGE  STATUS  PORTS
a1b2c3d4e5f6  nginx  Up 2 minutes  0.0.0.0:8080→80
9f8e7d6c5b4a  redis  Up 5 minutes  6379/tcp
```

Gracefully halt a container

```
docker stop <name|id>
```

- Sends a stop signal
- Lets the app shut down cleanly
- Container can be restarted

```
$ docker stop web
```

```
web
```

```
$ docker ps
```

```
CONTAINER ID IMAGE STATUS
```

```
# web no longer listed
```

Remove a container

```
docker rm <name|id>
```

! Stop it first

-f force-remove a running one

! Data without a volume is lost

```
$ docker stop web
```

```
$ docker rm web
```

```
web
```

```
# container removed
```

A recipe for your image

A Dockerfile is a plain-text file with step-by-step instructions. Docker reads it top to bottom and builds an image you can run anywhere.

- Repeatable and version-controlled
- Each line is a build step
- Steps are cached as layers

```
# Dockerfile
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r reqs.txt
CMD ["python", "app.py"]
```

Read it like a story

FROM Choose a base image to start from

WORKDIR Set the working directory inside the image

COPY Add your application files

RUN Install dependencies during the build

CMD Define what runs when the container starts

A few more you will use

ENV

Set environment variables inside the image.

EXPOSE

Document which port the app listens on.

ARG

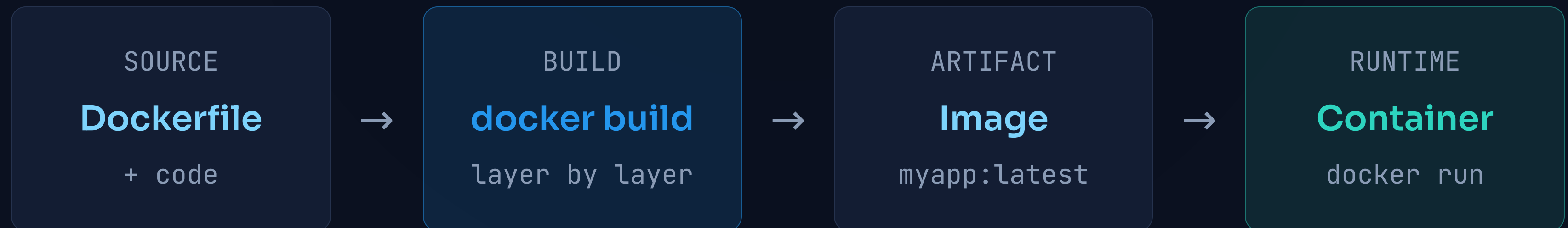
Pass build-time variables to the build.

ENTRYPOINT

Fix the main command; CMD becomes its args.

From Dockerfile to running app

```
docker build -t myapp .
```



Build it, then run it

- 1 Write the Dockerfile
- 2 Build a tagged image
- 3 Run it with a port mapping
- 4 Open it in the browser

```
$ docker build -t myapp .  
Successfully tagged myapp:latest  
  
$ docker run -p 8080:5000 myapp  
Running on http://0.0.0.0:5000
```

Where Docker shines

Microservices

Package each service independently and scale it on its own.

CI/CD Pipelines

Build and test in clean, identical environments every time.

Local Development

Spin up databases and tools without polluting your machine.

Cloud Deployment

Ship the same image to any cloud that runs containers.

The same image runs everywhere

AWS

ECS & Fargate

EKS (Kubernetes)

ECR registry

Google Cloud

Cloud Run

GKE (Kubernetes)

Artifact Registry

Azure

Container Apps

AKS (Kubernetes)

Container Registry

Watch out for these

Storing data in containers

Use volumes, or your data vanishes on removal.

Huge images

Pick slim base images and clean up build steps.

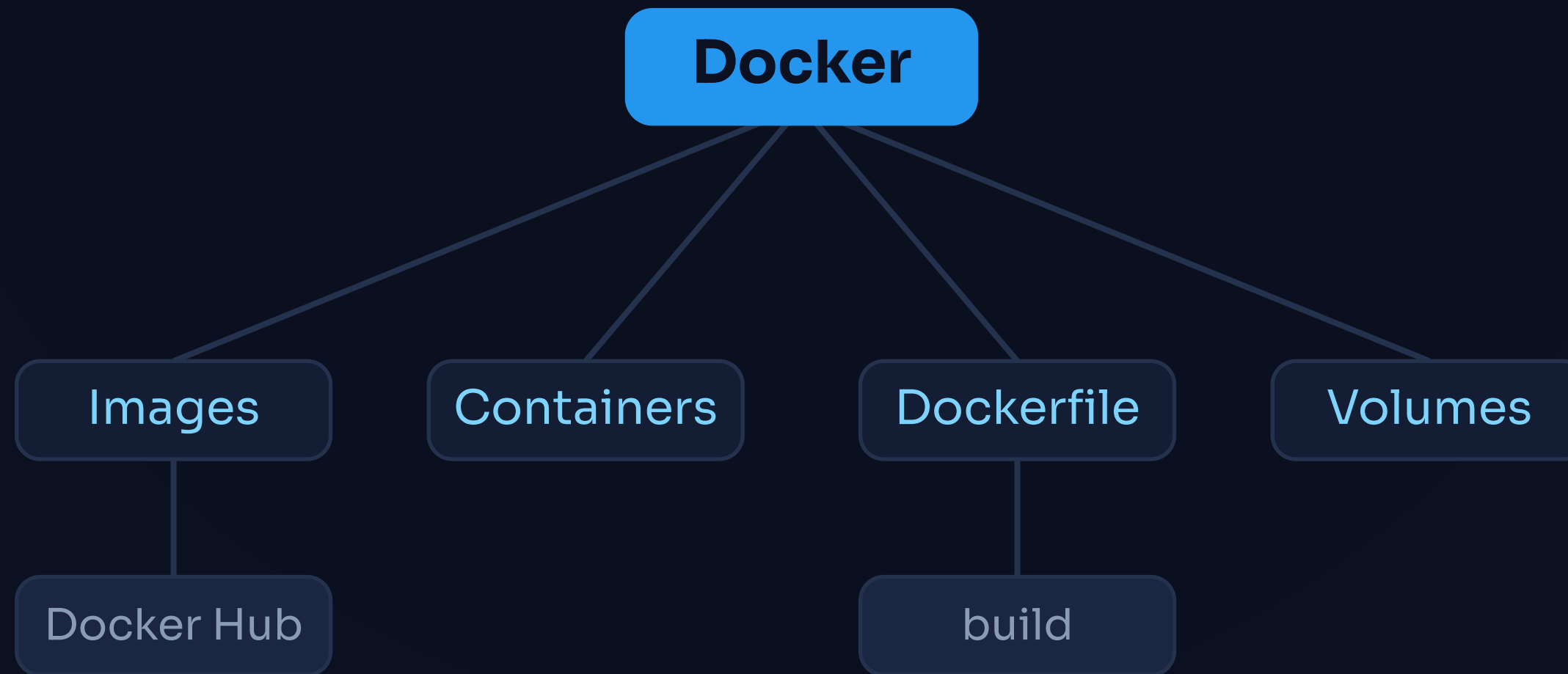
Running as root

Create a non-root user for safer containers.

Using latest everywhere

Pin versions so builds stay reproducible.

How it all connects



Test yourself

Q1 What is the difference between an image and a container?

Q2 Which command publishes a container port to the host?

Q3 Why do we use volumes instead of writing inside a container?

What to remember

Consistency

Containers package the app and its dependencies to run anywhere.

Images vs Containers

An image is the blueprint; a container is a running instance.

Dockerfile

A repeatable recipe that builds your image step by step.

Volumes

Keep data safe and persistent beyond the container lifecycle.

■ THANK YOU

Questions & Discussion

Let's talk through anything from images and containers to Dockerfiles and deploying to the cloud.



Imran Sayyed

Docker Fundamentals & Containerization